Typed but Not Guarded: An Automated Testing Pipeline and an Empirical Audit of Python TypeGuard Predicates

ANONYMOUS AUTHOR(S)

Type narrowing is a common pattern in gradually typed languages that refines types based on control flow semantics. Python's `TypeGuard` mechanism allows user-defined predicates to serve as claims for static type narrowing. This expressive mechanism, however, also delegates part of the checker's reasoning to arbitrary user code whose semantics are not validated. We present PPG, an automated audit pipeline that generates and executes property-based tests for `TypeGuard` predicates in off-the-shelf Python code. PPG combines static extraction of constraints with constraint-guided property-based testing, and checks whether the synthesized inputs satisfy the declared target type. The tool supports a practical subset of idiomatic Python guard code. We apply PPG to a corpus consisting of 19 Python projects containing 114 predicates.

Our audit reveals a substantial validation gap. Of the 114 predicates we studied, 11 are directly refuted by generated counterexamples, 52 resist effective automated validation, and 51 survive the audit without a found counterexample. We further classify validation-resistant predicates and audited failures into recurring families, showing that the problem is not limited to isolated bugs or obviously dynamic code. Instead, real-world `TypeGuard` usage exhibits a systematic validation gap: predicates are widely consumed as narrowing claims even when the evidence is obscure or obviously wrong. Together, these findings provide the first empirical audit of Python's user-defined narrowing ecosystem and show that real-world narrowing claims often lack the level of validation that current type checkers assume.

CCS Concepts: • **Software and its engineering → Software defect analysis**; *Dynamic analysis*; *Automated static analysis*; Language features; Software maintenance tools.

Additional Key Words and Phrases: Python, gradual typing, type narrowing, user-defined predicates, property-based testing, empirical software engineering

ACM Reference Format:
Anonymous Author(s). 2026. Typed but Not Guarded: An Automated Testing Pipeline and an Empirical Audit of Python `TypeGuard` Predicates. 1, 1 (June 2026), 26 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Gradual typing is spreading among dynamically typed languages [2, 5, 8, 32, 41, 43] and especially Python [1, 12, 19, 27, 29, 30, 44]. One important feature of these systems is type narrowing, which refines variable types based on control flow. Given the inherent dynamic nature of such languages and the growing complexity of the types, increasingly relies on user-written predicates. Python's `TypeGuard` lets developers define custom predicates whose `True` return values tells the typechecker to narrow a value to the claimed type; a `False` result has no corresponding meaning. This is convenient and expressive: programmers can encode project-specific notions such as "`x` is this schema field," "`y` is this token subtype," or "`z` is this cursor-like object" without awkward type-level encodings. But it also introduces a hidden assumption. Once a `TypeGuard` returns `True`, the checker trusts that the predicate body established the declared target type, even though that body is arbitrary user code and its acceptance condition is never checked.

XXXX-XXXX/2026/6-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Unsound narrowing example from `tinygrad/tinygrad`.

```
def all_int(t: Sequence[Any]) -> TypeGuard[tuple[int, ...]]:
    return all(isinstance(s, int) for s in t)
```

Generated test shape.

```
@given(t=st.one_of(
    st.lists(st.integers()),
    st.lists(st.integers()).map(tuple),
))
def check_path_1(t):
    assert all_int(t) is True
    assert runtime_matches_target_type(t, tuple[int, ...])
```

Concrete witness.

```
t = [1]
```

Why it passes	The guard checks only an elementwise property: every member of `t` is an `int`. It never checks that `t` is a `tuple`.
Why narrowing is unsound	The declared target is `tuple[int, ...]`, but the witness is the list `[1]`. It therefore falls outside the target type even though the predicate returns `True`.

Fig. 1. An introductory unsound `TypeGuard` from `tinygrad/tinygrad`.

Figure 1 demonstrates a simple example from `tinygrad/tinygrad` illustrating the issue. The body proves only an element-wise fact: every member of the `Sequence` `t` is an `int`. The declared target, however, is `tuple[int, ...]`. A list of integers can therefore satisfy the predicate and still drive control flow down branches that assume a tuple. This is unsound narrowing: a value satisfying the predicate can still fall outside the claimed target type.

The core problem here is not just that developers can make mistakes. It is that user-defined narrowing logic introduces a validation obligation, and the current ecosystem does not fulfill that obligation. To understand the impact of such a mismatch, we need to examine the validation gap between the narrowing claims of real-world `TypeGuard` predicates and the acceptance conditions they implement. This brings forth our preliminary research question:

PRQ: How many of the real-world `TypeGuard` predicates are not trustworthy?

Unfortunately, accurate checking requires significant changes to the typechecker infrastructure, such as occurrence typing [48] or set-theoretic types [7]. Few gradual typecheckers have such support. Indeed, a recent effort to benchmark language designs for type narrowing found that lack of support for typechecking user-defined predicates (such as `TypeGuards`) is a leading cause of unsoundness [20].

Hence, rather than attempting to fully verify all `TypeGuard` predicates, we turned to an empirical approach. We collected a corpus set of 114 predicates from 19 open-source Python projects that each have more than 10,000 stargazers on GitHub. We also implemented Parsing-Predicates-to-Generate (PPG), a hybrid tool that combines static analysis and property-based testing to automatically validate the semantic soundness of `TypeGuard` predicates in off-the-shelf Python code.

PPG treats each `TypeGuard` predicate $P(x)$ as a claim: *if* $P(x)$ *returns* `True`*, then* x *should satisfy the declared target type.* Instead of relying on hand-written tests, PPG synthesizes *property-based tests* by combining static extraction of `return True` path constraints with constraint-guided input generation. It then checks whether these accepted values actually inhabit the declared target type,

making PPG a practical automatic tester for real-world `TypeGuard` code. For the example above, it extracts that `t` is a `Sequence` with all-`int` elements, uses this fact to guide the Hypothesis PBT framework, and produces the test in Figure 1 (omitting boilerplate). The resulting strategy explores `lists` and `tuples` of integers and quickly finds counterexamples such as `list([1])`.

With this automated audit tool, we can ask a more settled version of the preliminary research question:

RQ1: What audit outcomes does PPG obtain on real-world TypeGuard predicates?

We then carried out a empirical audit by applying PPG to the corpus, which reveals a systematic validation gap. Of the 114 predicates, 11 are *directly refuted* by generated counterexamples, 52 are *validation-resistant*: they inhabit patterns that resist effective automated validation, and only 51 *survive the audit without a found counterexample*. Even those PASS outcomes remain provisional because PPG does not cover every execution path. In other words, most predicates in the evaluated set therefore lack strong positive evidence. This result motivates our next two research questions:

RQ2: Why do many real-world `TypeGuard` predicates resist effective automated validation?
RQ3: What kinds of unsoundness are exposed by witnessed counterexamples?

To answer these questions, we manually reviewed and classified every validation-resistant and directly refuted predicate. We examined all such predicates one by one to identify the pattern responsible for the outcome. We then grouped these recurring patterns into taxonomies, resulting in 3 taxonomies for validation-resistant predicates and 5 taxonomies for directly refuted predicates. The validation-resistant cases are not random leftovers. Instead, they cluster into recurring families such as *helper-dependent guards*, *guards with unsupported constraint syntax*, and *guards with runtime non-local dependencies*. The directly refuted cases likewise reveal recurring semantic mistakes, including *weak discriminators*, *duck-typed nominal claims*, *container kind mismatches*, *shallow protocol checks*, and *class-spoofing mismatches*.

Taken together, these results answer the three questions that motivate the paper. RQ1 shows a broad lack of strong validated guarantees for real-world narrowing claims. RQ2 explains why this gap persists: many predicates are written in forms whose acceptance condition is difficult to audit automatically. RQ3 shows that when automated auditing succeeds, it uncovers concrete, recurring mismatches between acceptance conditions and declared target types. This combination justifies an empirical account of the validation gap and an underlying automated audit pipeline that could also help narrow the gap.

Our contributions are:

- PPG, an automated audit pipeline that combines static success-path constraint extraction with constraint-guided property-based testing to validate `TypeGuard` predicates in off-the-shelf Python code.
- An empirical audit of 114 real-world `TypeGuard` predicates from 19 open-source Python repositories, showing that real-world `TypeGuard` usage exhibits a substantial validation gap.
- A manual characterization of that gap, including a taxonomy of validation-resistant predicates and a taxonomy of witnessed unsoundness patterns that explain both why automated validation is blocked and what semantic mistakes it exposes in practice.

2 BACKGROUND

The question of how to typecheck Python has been attracting significant interest over the past decade, since the creation of an official syntax for annotations [52]. Today, Python comes with a rich set of built-in types (but no official typechecker) [37] and a repository of types for common libraries [35]. There are several typecheckers available, from venerable tools such as Mypy [44] and

Pytype [19] to newer additions such as Pyright [30], Pyrefly [29], and ty [1]. The Python community is also supported by a Typing Council of experts to consult on type system designs [36].

`TypeGuard` and its close relative, `TypeIs`, are two built-in Python typing primitives. They are intended to be used only as the return type of a function (or method). The function must return a Boolean, and comes with an additional side effect when called in a conditional expression:

- If `f` returns a `TypeGuard[T]`, then the statement `if f(x): e0 else: e1` can assume that `x` has type `T` in `e0`.
- If `f` returns a `TypeIs[T]`, then the statement `if f(x): e0 else: e1` can assume that `x` has type `T` in `e0` and that `x` does not have type `T` in `e1`.

We study only `TypeGuard` in this paper because it was added to Python earlier and has broader adoption in our corpus, and also because `TypeIs` has stricter semantics [56] and is checked more closely (but still very limited) by typecheckers [20], thus exposing a smaller potential hole.

Many other gradual languages support user-defined predicates as well. In Flow [28], the return type `: implies x is T` has the same effect as a Python `TypeGuard`, and the return type `: x is T` acts as a `TypeIs`. TypeScript [31] supports only the `TypeIs` variant. Typed Racket [47] and Typed Clojure [3] expose a formula-like language of predicates that can refine the types of any argument to a user-defined function, going well beyond `TypeGuard` or `TypeIs` alone. CDuce supports only `TypeIs` [6, 7]. For the most part, implementations of `TypeGuard` are unsound in the sense that they do not check the side effects of a user-defined function. A function with return type `TypeGuard[T]` can simply return true instead of validating that its input matches the type `T`. As shown in the If-T Benchmark [20], none of the Python typecheckers validate user-defined predicates. TypeScript does no validation. Flow validates only simple predicates (as of writing, it can handle a ternary expression but not an `if`/`else` statement). Typed Racket, Typed Clojure, and CDuce have much more extensive support for checking, but at the cost of a steep learning curve.

The context for this paper is thus that Python typecheckers provide no protection against faulty `TypeGuards`, and this can lean to unsoundness. To give a simple example, a function that always returns true and has the return type `TypeGuard[int]` is able to cast any type to an `int` type. Our goal is not to typecheck `TypeGuards`, but rather to provide lightweight checking and testing that developers can opt in to, after passing the basic requirements of their preferred typechecker. Our tool PPG is not tied to any specific Python typechecker by design to facilitate adoption.

3 STUDY DESIGN AND DATASET

Our evaluation dataset is drawn from open-source Python projects on GitHub. We began with an initial list of 68,655 repositories generated by Mozilla's Agnostic GitHub API (`agithub`) to include repositories that used Python, that had over 80 stars, that *were not* forks of another repository, and that included one of the following files: `mypy.ini`, `pyproject.toml`, `setup.cfg`, or `config`. These specific filenames are supported by `mypy` for configuration options. We filtered the list to remove missing repositories in January 2026 by cloning each repo. Then we drastically reduced the set by performing a text-based search for the string TypeGuard[, the prefix of a `TypeGuard` type annotation. This resulted in 352 projects. We manually inspected the results to arrive at 256 projects with actual `TypeGuard` declarations. As this set was too large to analyze in detail, we focused the search on projects with over 10,000 GitHub stars (we acknowledge this is an imperfect filter [26]). The final set consists of 19 projects. These projects contain 114 `TypeGuard` declarations total and consist of over 3 million lines of Python code.

The repositories cover a mix of Python libraries and tools. This diversity is important because `TypeGuard`, appears in very different settings: compiler or typechecker infrastructure, web and UI frameworks, data-processing libraries, interactive tooling, schema-processing code, and general

utility code. The resulting dataset is therefore not merely a collection of toy examples; it is a cross-section of real projects that rely on custom narrowing logic in production code.

The unit of analysis is a single `TypeGuard` predicate. For each predicate, we attempt to run the audit pipeline described in Section 4. In the current dataset, all predicates that are either directly refuted by witnessing counterexamples or resisting automated validation are manually classified into a reviewed category. The review process is described in more detail in Section 4.3.

4 PPG: AUTOMATED TEST GENERATION AND AUDIT PIPELINE

PPG is a working tool for automatically testing the semantic soundness of `TypeGuard` predicates in ordinary Python source code. It combines static extraction of `return True` path constraints with constraint-guided property-based testing. This approach enables it to take in the unmodified Python code, analyze the predicate body, synthesize a property-based test, execute it, and record the outcome. The overall architecture of the PPG pipeline is shown in Figure 2.

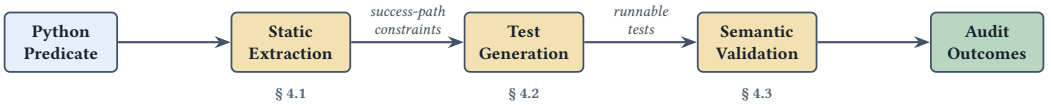

Fig. 2. Overview of the PPG pipeline.

At a high level, PPG implements the following workflow:

(1) **Front-end discovery and guarded-value identification.** PPG identifies functions and methods annotated with `TypeGuard` and selects the guard subject from the parameter list.
(2) **Static extraction of `return True` path constraints.** PPG performs an under-approximating analysis of the predicate body, recording only constraints that hold on reachable paths that can return `True`.
(3) **Strategy synthesis for property-based testing.** The extracted constraints are lowered into Hypothesis strategies that guide generation toward values guaranteed to be accepted by the predicate.
(4) **Execution of the semantic validation.** PPG runs generated tests through the original predicate to ensure they return `True`, then compares them against the declared target type.
(5) **Outcome classification.** PPG reports whether the predicate was refuted by a counterexample, survived the test campaign without a found counterexample, or resisted effective automated validation.

We use the predicate in Listing 1 as a running example throughout this section. Its target type `UserRow` requires a dictionary with `kind == "user"`, an integer `id`, and a string `name`. The predicate, however, checks only that the input is a dictionary with the right discriminator and the keys `id` and `name`. A value can therefore satisfy the predicate while still violating the declared target type, which is exactly the mismatch PPG is designed to expose.

```python
class UserRow(TypedDict):
    kind: Literal["user"]
    id: int
    name: str

def is_user_row(obj: object) -> TypeGuard[UserRow]:
    return (
        isinstance(obj, dict)
        and obj.get("kind") == "user"
        and "id" in obj
        and "name" in obj
    )
```

Listing 1. Running example used throughout Section 4.

4.1 Static Extraction of Constraints

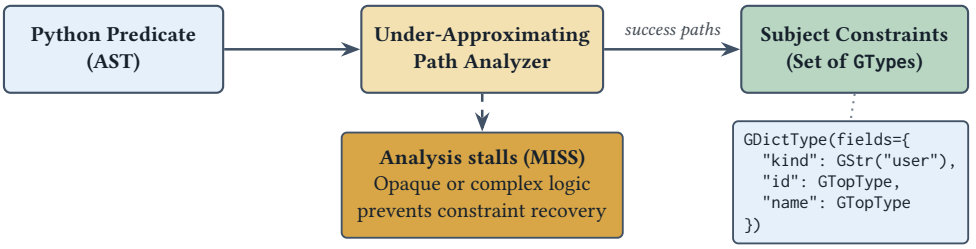

Fig. 3. Constraint extraction process.

The first stage of the PPG pipeline is the static extraction of path constraints from the predicate body. The goal is to build a formal representation of the conditions that must hold for the subject of the `TypeGuard` to reach a `return True` statement. As shown in Figure 3, this analysis explores the predicate's control flow and summarizes the requirements on the guarded value.

PPG supports a substantially expressive and idiomatic Python syntax subset, as described in Table 1. This ensures that the current prototype is able to analyze a large proportion of off-the-shelf Python `TypeGuard` predicates. At the statement level, PPG handles `if`/`elif`/`else`, `assert`, limited `for`-loops, and various forms of assignment. At the level of expression, PPG supports path forms such as attribute access, indexing, and other ways of accessing object properties such as `getattr(obj, "name")`. Particularly, expressions that carry narrowing semantics are supported; this includes common narrowing primitives such as `isinstance`, `issubclass` and `hasattr`, membership tests, equality and inequality against literals, truthiness and length checks, exact-binding identity tests, and calls to other `TypeGuard` predicates including recursive calls to self. As the reader will see in Section 5.2, the reasonable coverage of supported syntax suggests the presence of complex or obscure patterns in `TypeGuard` predicates that fall out of this scope.

As one essential goal of PPG is to synthesize inputs that guarantee acceptance, it should retain only the facts justified by the `return True` while keeping maximum coverage in that range. Consequently, unlike typical type-based static analyses that over-approximate the domain of values, PPG deliberately under-approximates it, thus generating a variety of values that must fall into the approximated domain. In the running example, that means inferring that the value corresponding to key `"kind"` of `obj` can only be the literal `"user"` instead of a wide string type, while leaving `id` and `name` unconstrained. More generally, on encountering unsupported semantics, PPG treats

Table 1. Representative Python syntax currently supported by PPG.

Category	Supported fragment
Statements	`if`/`elif`/`else`, `assert`, `return` of an analyzable condition, simple assignment, tuple/list unpacking, and restricted `for`-loops used for universal checks.
Access paths	Names, attribute chains, indexing, `getattr(obj, "attr")`, `cast(T, x)`, and `d.get("k")` when it can be rewritten to a keyed lookup; local aliases to such paths are tracked.
Narrowing tests	`isinstance`, `issubclass`, `hasattr`, membership tests, equality/inequality against literals, `is`/`is not` tests against `None`, booleans, or exact bindings, plus truthiness and selected `len(...)` checks.
Composition and helpers	Boolean composition with `not`, `and`, `or`, ternary expressions, and calls to previously analyzed or imported `TypeGuard` predicates, including recursive self-calls.
Iterables and comprehensions	`all(...)` with a single generator or list comprehension, plus analyzable iteration over lists, tuples, dictionaries, `keys()`/`items()`, and values typed as common iterable or container ABCs.
Conservative boundary	Arbitrary calls, `any(...)`, `while`, `with`, `try`/`except`, `match`, rich comparisons such as `len(x) > 0`, and more complex comprehensions fall outside the current fragment; affected success paths are dropped.

the relevant values conservatively by tainting them as `Unknown`. If an unsupported expression or statement taints a success path with `Unknown`, PPG drops that path from the final constraint set instead of inventing facts it cannot justify. Following this approach, PPG generates reliable tests: values generated according to the recorded constraints must lead to `True` for the return value. This matches the one-way semantics that `TypeGuard` is supposed to satisfy as described in Section 2. By guaranteeing generation of `True`-leading inputs, PPG can search directly for counterexamples witnessing unsound narrowing.

To bridge the gap between Python's flexible runtime behavior and the static needs of representing these constraints while being flexible and generic, we designed a constraint lattice that is not tied to the internal type system implementation of any specific typecheckers. This representation is called `GType` (`G` for "guarded"). As summarized in Table 2, it supports a rich range of types. By reconstructing these constraints from the predicate's code, PPG infers the constraints shaped by the "accepted region" of the input space. In the running example, that accepted region is the weaker dictionary shape recovered in Figure 3, not the full `UserRow` target type.

However, this extraction process can fail to produce a precise constraint set, leading to an MISS outcome. Such "analysis stalls" typically occur when the predicate's logic is too opaque or complex for local static analysis to summarize. This includes reliance on external helper functions whose bodies are not inspected, control flow that depends on runtime non-local state, or complex generator-based checks that do not collapse to simple type-based constraints. In these cases, PPG conservatively marks the path as undetermined rather than risking an unsound approximation. As we show in Section 5.2 by providing a detailed taxonomy of these validation-resistant patterns, this reveals not simply a limit of the tool, but a significant barrier to automated validation of `TypeGuard` predicates.

Table 2. Representative forms in the GType language used by PPG.

Form	Meaning
Lattice markers	`GTopType`, `GBottomType`, and `GUnknownType` represent unconstrained values, contradictions, and analysis-tainted facts that are handled conservatively.
Literal and identity constraints	Literal types that encode exact strings, integers, booleans, `None`, and floats, while `GExactValueType` captures specific non-primitive runtime values.
Nominal and structural objects	`GInstanceType` records nominal class membership and can attach field facts (e.g., an instance whose `kind` attribute must equal `"wanted"`); `GObjectType` records attribute-only structural shapes without committing to a nominal class.
Containers	`GListType` and `GTupleType` record per-index constraints plus a residual; setting the residual type to `GBottomType` expresses exact length. `GDictType` similarly combines distinguished key facts with residual key and value types for remaining entries.
Unions and recursion	`GUnionType` represents alternative accepted shapes, while `GRecursiveType`/`GVarType` encode self-referential structures such as linked lists or nested trees.

4.2 From Constraints to Generated Tests

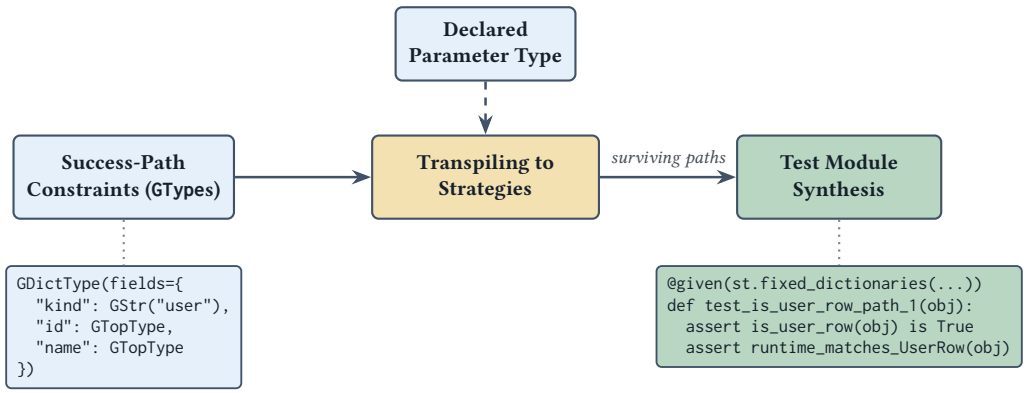

Fig. 4. Test generation process.

The previous stage does not yet produce tests; it produces the constraints describing the predicate's accepted region, that is, the shape of values sufficient to drive execution to some `return True` path. They are the values that cause the typechecker to trust the narrowing claim. We then need to transpile these shape constraints to test code that is actually executable. The transition from recovered constraints to runnable tests is illustrated in Figure 4.

In the running example, this means generating dictionaries that look "user-shaped" enough as checked in the body of `is_user_row` for it to accept them. PPG therefore transpiles each recovered success-path constraint into a path-specific Hypothesis strategy. Literal constraints become singleton strategies; conjunctions over container structure become strategies that build

values with the required shape; and disjunctions become unions of strategies. In the running example, the literal fact `obj["kind"] == "user"` becomes a fixed field in the generated dictionaries, while the unconstrained `id` and `name` positions become generators over arbitrary values. More generally, a constraint requiring a dictionary with distinguished fields becomes a strategy that always produces dictionaries containing those keys, whereas a constraint describing "a list whose elements are strings" becomes a list strategy parameterized by a string generator. Structural object constraints are realized by synthesizing objects with the required attributes, and nominal class constraints use either built-in generators for common runtime types or project-specific constructors when available. Before emitting the final strategy, PPG intersects each success-path description with the predicate's declared parameter type so that generated inputs not only are acceptance-directed but also pass parameter typechecking. In the running example, the parameter type is `object`, so this intersection is trivial; in repository code it often rules out values that match a recovered path summary but could not be passed to the original predicate.

Each surviving success path is then woven into a runnable test. PPG emits a pytest test module that imports the original source file, resolves the `TypeGuard` function or method under test, and binds the path-specific strategy to the guarded parameter. For class method predicates, the harness separately generates or constructs a receiver object, so the test still invokes the original method rather than a simplified model of it. When the tool cannot generate essential context, such as additional required parameters or non-synthesizable instances of classes, it does not fabricate one; those cases are instead handed off to the programmer to supply the missing generation strategy. In our corpus this fallback was rare: after the repository dependencies were installed, only 3 predicates required such providers, realized by 3 entries across 2 generated test modules.

The test body performs the audit end to end. For every generated value, it first calls the original predicate and asserts that the result is `True`; this checks that the generated input really does inhabit the recovered accepted region. It then checks the same value against a runtime interpretation of the declared `TypeGuard` target type. If the predicate accepts the value but the target-type check fails, the generated test has found a counterexample witnessing unsound narrowing. If no counterexample is found within the search budget, the predicate receives only provisional positive evidence: the generated values exercised accepted executions, but they did not refute the claim.

For `is_user_row`, a dictionary such as `{"kind": "user", "id": "abc", "name": []}` is exactly such a counterexample: it satisfies the recovered success-path constraint and makes the predicate return `True`, but it does not satisfy the declared `UserRow` type because the field values have the wrong types.

4.3 Targeted Checking and Outcomes

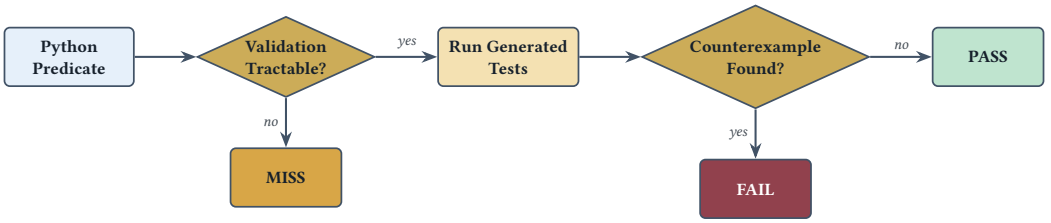

Fig. 5. Audit outcome classification process for a predicate.

The predicate audit and outcome classification process is illustrated in Figure 5. For each generated input, we used the Pytest [23] Python test runner. There are three possible outcomes:

- **PASS**: PPG generated and ran tests but found no counterexample.
- **FAIL**: PPG generated a value for which the predicate returned `True` even though the value failed to match the declared target type.
- **MISS**[1]: PPG could not provide effective automated validation, typically because the predicate's `True`-returning paths could not be recovered precisely enough from the supported syntax and analysis model.

These outcomes should be read as states for evidence finding, not as a scoreboard that adjudges the soundness of the predicate. Most importantly, PASS means only that PPG did not find a witness counterexample against the claimed type. In the meanwhile, MISS indicates that the predicate influences narrowing in a form that escapes effective automated validation by the current tool, and FAIL is strong negative evidence about the predicate.

After the automated run, we manually review those that received an MISS or FAIL outcome and assign them to semantic categories. This review is what lets us separate concrete unsoundness from validation resistance and steer PPG from a bug-finding tool into an empirical instrument for understanding how real projects use `TypeGuard`.

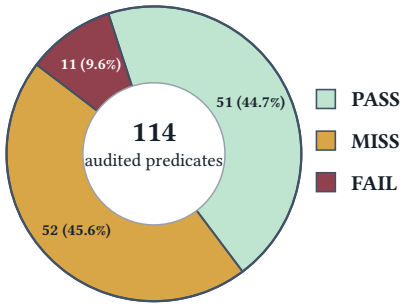

Fig. 6. Overall distribution of audit outcomes across the evaluated `TypeGuard` predicates.

A single researcher assigned labels after reviewing the predicate, its target type, any generated witness, invoked helpers, and the relevant local repository context. When multiple factors appeared relevant, we recorded one primary label intended to capture the dominant actionable obstacle to validation; for MISS cases, this assignment followed a fixed priority rule (helper dependence, then unsupported syntax, then external mutable-state dependence), while FAIL cases in the analyzed corpus did not require tie-breaking. Accordingly, the taxonomy reports primary descriptive labels rather than complete causal attributions.

4.4 Validating PPG as an Audit Instrument

As stressed above, PPG is not a verifier but an empirical audit instrument. Its outcome labels are therefore meaningful only if three operational contracts hold: synthesized strategies must generate values that the original predicate actually accepts, the runtime target-type checker must classify concrete values correctly on its supported fragment, and recorded FAIL outcomes must be replayable rather than artifacts of a single search run.

First, PPG is acceptance-directed by construction: it under-approximates the predicate's `return True` region and emits tests whose first assertion re-checks that the original predicate returns `True` on every generated input. Across repeated runs of all non-MISS audits, we did not observe any generated value fail this acceptance check. In the current evaluation, this covers all 62 predicates whose audits completed without MISS (51 PASS, 11 FAIL). We also regression-tested the constraint analyzer with 2,583 lines of tests comprising 97 unit tests over the supported extraction patterns. These results do not show completeness, but they do support the narrower claim needed here: on the tractable fragment, PPG's synthesized strategies generate values from the predicate's accepted region rather than merely nearby inputs.

[1]The raw outcome kind produced by Pytest is `XFAIL`; we rename it in the paper to distinguish from FAIL.

Second, PPG reports a counterexample only relative to a documented runtime interpretation of the declared `TypeGuard` target. The runtime target-type checker distinguishes supported matches, supported mismatches, and unsupported target forms; FAIL is reported only for accepted values that fall into the second category. In its current implementation, the checker supports the common types corresponding to the constraint forms exercised in our corpus, including nominal types, `Any`, `None`, primitive `Literals`, `Union`, common container generics, tuples, `type[T]`, certain aliases, and `TypedDict`; it does not claim support for more advanced forms such as protocols, overloaded callables, or complex `TypeVar`-based targets. The checker is separately regression-tested by 850 lines of tests comprising 29 unit tests over this supported fragment.

Finally, we validated every recorded FAIL by replay. We reran all 11 FAIL cases in each repository's local test environment with Hypothesis debug output enabled, using five seeds and two budgets (default and `max_examples=1000`) where supported, with the example database disabled so cached witnesses could not be reused. All 11 cases reproduced. For every case, the median number of generated examples to first witness was 1 or 2, and increasing the budget did not materially change the outcome. This indicates that the current FAIL witnesses are stable and easy to rediscover rather than flaky artifacts of a particular run.

These checks do not turn PASS into proof, nor do they remove the incompleteness represented by MISS. They do, however, establish the narrower claim required for the empirical study: when PPG reports FAIL, it is finding reproducible counterexamples from executions that the original predicate itself accepts, judged against a documented runtime interpretation of the declared `TypeGuard` target.

5 AUDIT RESULTS

The first result of the audit is the overall evidence distribution. Out of 114 audited predicates, 51 are labeled PASS, 11 are labeled FAIL, and 52 are labeled MISS. Interpreted through the lens of evidence, this means the following:

- **11/114 (9.6%)** are directly refuted by generated counterexamples.
- **52/114 (45.6%)** resist effective automated validation and remain semantically unvalidated under our method.
- **51/114 (44.7%)** survive the audit without a found counterexample.

This overview already suggests two conclusions, one encouraging and one cautionary. The first is encouraging because it shows that PPG is a practically useful automatic tester for user-defined narrowing predicates: it can recover enough semantic structure from many real predicates to synthesize targeted tests guiding the automatic search for real bugs in existing libraries without requiring rewritten code and with almost no hand-crafted generation support, demonstrating that automatic property-based testing of `TypeGuard` predicates is feasible on a non-trivial portion of off-the-shelf predicates. The second conclusion is troubling: 63/114 predicates (55.3%) end the process either directly refuted (FAIL) or still lacking effective automated validation (MISS). Only a minority survive the generated tests without a found witness against them, and even those survivors are not certified correct. This means the ecosystem contains many `TypeGuard` predicates whose narrowing claims are either falsified or blindly trusted without strong validating evidence especially given their complicated patterns; together, these non-PASS outcomes expose a validation gap between the power of user-defined narrowing and the evidence available to support it. In the following subsections, we address the overall reliability of `TypeGuard`, and discuss the taxonomy of MISS and FAIL cases correspondingly along with small sets of representative case-study families.

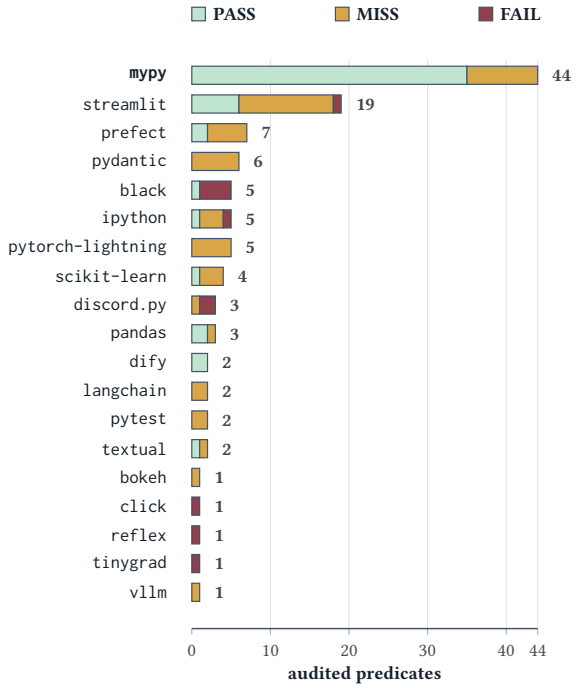

Fig. 7. Repository-level distribution of audit outcomes.

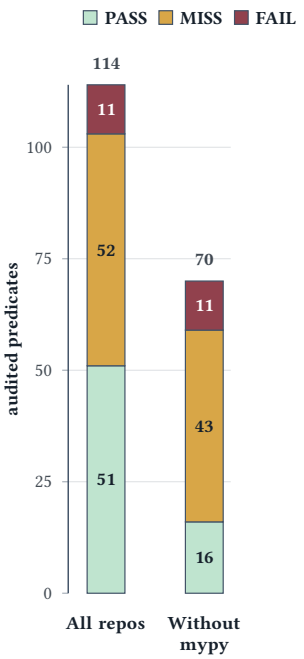

Fig. 8. Outcome counts with and without mypy.

5.1 RQ1: The Reliability of Real-World `TypeGuard`

Our first research question asks what our empirical audit reveals about the overall reliability picture of real-world `TypeGuard` predicates. The answer is that strong and straightforward positive evidence for the narrowing claims declared by these `TypeGuard` predicates is relatively rare. The evaluated ecosystem contains both witnessed unsoundness and substantial validation opacity, and these phenomena are spread across many repositories rather than concentrated in a single isolated project.

At the repository level, FAIL occurs in 7 repositories and MISS occurs in 14 repositories. By contrast, PASS appears in only 9 repositories. Moreover, 6 repositories in the evaluated set contain only MISS cases and no surviving audited predicates at all. Only one repository in the current dataset (`langgenius/dify`) is PASS-only, and it contributes only 2 predicates. This asymmetry suggests that the main story is not "most predicates validate cleanly, with a few hard cases on the side." The real story is that many predicates either fail or resist effective validation.

The distribution is also uneven. The repository of `python/mypy`, a Python typechecker itself, contributes 44/114 predicates (38.6%) and 35/51 PASS outcomes (68.6%). This concentration matters because it makes the overall outcome distribution somewhat more optimistic than the rest of the dataset. A simple sensitivity view excluding mypy strengthens the overall result rather than weakening it: outside mypy, the dataset contains 43 MISS, 11 FAIL, and only 16 PASS outcomes. In that view, 54/70 predicates (77.1%) are either directly refuted or remain semantically unvalidated. We emphasize this result cautiously, because the dataset is not balanced across repositories, but it supports the qualitative conclusion that the validation gap is not an artifact of a single large project.

One thing worth noting is that reliability in this study is not synonymous with "accepted by the tool." A PASS only means that our audit did not find a counterexample under the available search budget and the supported syntax subset. It does not mean that the predicate is semantically correct in all executions or under all interpretations of the target type. The best way to read RQ1 is therefore this: an empirical audit of real-world `TypeGuard` predicates reveals that a large fraction of them lack strong validating evidence, either because they are directly contradicted or because effective automated validation is blocked. This result reveals that in the evaluated set, the trust that current typecheckers place in user-written narrowing code often exceeds the evidence available to justify it.

Figures 7 and 8 render the point visually: the repository-level distribution is highly uneven, and excluding mypy removes many PASS cases without softening the broader reliability concern.

RQ1 Result: Most of the predicates (55.3%) are either directly contradicted or resist effective validation, and excluding mypy raises this share to 77.1%.

5.2 RQ2: Reasons for Resisting Effective Validation

Our second research question asks what the MISS outcomes mean. We do not interpret MISS as a mere coverage failure of PPG. Instead, we ask why so many predicates resist effective automated validation and what that resistance reveals about real-world `TypeGuard` usage.

Across the 52 MISS cases, the largest class is *helper-dependent guards* with 29 cases (55.8%), followed by *guards with unsupported constraint syntax* with 12 cases (23.1%), and *guards with runtime non-local dependencies* with 11 cases (21.2%).[2] The main barrier is therefore not simply "Python is dynamic." Rather, validation is often blocked at three specific analysis boundaries: helper calls opaque to validation, guard syntax we do not yet support, and dependence on values or state external to the guarded argument.

A family-level view helps avoid clone inflation. When same-source same-class siblings are grouped together, the 52 MISS rows collapse to 31 source-level families. In that deduplicated view, *helper-dependent guards* remains the largest class with 17 families (54.8%), *guards with unsupported constraint syntax* contributes 11 families (35.5%), and *guards with runtime non-local dependencies* shrinks to 3 families (9.7%). The breadth of occurrence reinforces this picture: *helper-dependent guards* appears in 11 of the 14 repositories containing at least one MISS, *guards with unsupported constraint syntax* in 9, and *guards with runtime non-local dependencies* in only 3. Thus, runtime non-local dependence is real but concentrated. Much of its row-level weight comes from dense sibling clusters in `mypyc/ir/rtypes.py` from `python/mypy` and `pydantic/_internal/_core_utils.py` from `pydantic/pydantic`, while helper dependence and unsupported syntax are more broadly distributed across the ecosystem. Figures 9 and 10 summarize this shift visually. Helper dependence remains the dominant barrier under every aggregation, but runtime non-local dependence shrinks sharply once same-source siblings are collapsed and remains concentrated in only a small repository cluster.

Helper-dependent guards are especially revealing. Here the local `TypeGuard` body may look simple, but the semantic substance of the check is delegated to helper functions, methods, or library utilities whose behavior our local extraction cannot soundly summarize. This is reasonable software engineering practice: real projects factor recognition logic into reusable helpers. But it weakens auditability. The checker consumes the narrowing result at the call boundary, while the evidence that would justify that result is hidden behind additional calls that are harder to inspect, summarize,

[2]For reproducibility, these correspond to the internal audit labels `T4_OPAQUE_HELPERS`, `T3_CONSTRAINT_COMPLEXITY`, and `T5_NONLOCAL_INPUT_COUPLED_PREDICATE`.

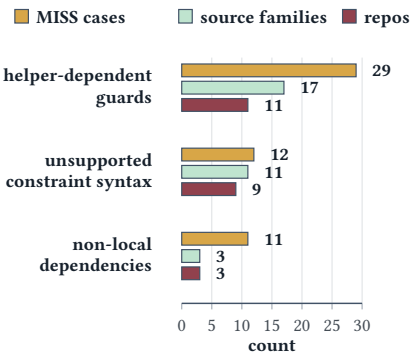

Fig. 9. Three views of the MISS taxonomy.

Fig. 10. Repository-level spread of the three MISS classes.

and test automatically. This is especially concerning due to the dynamic nature of Python: an opaque function could change the shape of involved values drastically.

MISS Case Study. Representative helper-dependent pattern from `streamlit/streamlit`.

```python
def is_plotly_chart(obj: object) -> TypeGuard[Figure | list[Any] | dict[str, Any]]:
    return (
        is_type(obj, "plotly.graph_objs._figure.Figure")
        or _is_list_of_plotly_objs(obj)
        or _is_probably_plotly_dict(obj)
    )
```

Local acceptance condition	One of three helper-based recognizers returns `True`; the local body itself contributes little more than a disjunction over helper results.
Hidden evidence	`is_type` performs string-based runtime type probing, while the other two helpers inspect list elements and dict keys/values through further helper calls.
Why extraction stalls	A sound summary would require interprocedural reasoning over several helper bodies plus library-specific naming conventions, so the accepted region is no longer a compact local constraint over `obj`.
Why this matters	The checker still narrows to `Figure | list[Any] | dict[str, Any]` at the call boundary, even though the evidence justifying that union is distributed across helper APIs.

Fig. 11. Helper-dependent guards in `streamlit/streamlit`.

Case Study: Opaque Helper Chains in `streamlit/streamlit`. The predicate `is_plotly_chart` in `streamlit/streamlit` illustrates this pattern clearly. The local guard body is short and plausible, but almost all of its semantic content is delegated to helper predicates and runtime type recognizers defined elsewhere. The resulting `TypeGuard` is reusable and convenient, yet the accepted region is no longer spelled out in a locally auditable form. This case is representative not because the helpers are exotic, but because the decomposition is normal engineering. `streamlit/streamlit` contains a dense cluster of sibling guards in the same file, and `langchain-ai/langchain` plus

PrefectHQ/prefect show the same pattern for async-recognition predicates. The validation problem is therefore structural: the more recognition logic is factored behind helper boundaries, the less the local TypeGuard body carries an auditable witness for the claimed narrowing.

The second broad barrier is *guards with unsupported constraint syntax*. These predicates resist validation not because their semantics are hidden elsewhere, but because the local syntax itself expresses conditions that do not collapse to ordinary narrowing facts. try-except turns acceptance into a statement about the non-occurrence of an exception. Generator-based any introduces existentially quantified conditions over elements or traversed sub-objects. Top-level negation asks for complement reasoning, which in a type-theoretic reading amounts to negation types. These are not merely missing parser cases: even a stronger typechecker would need effect-sensitive refinements, existential quantifiers, or negation machinery to internalize such checks as stable narrowing evidence.

MISS Case Study. Representative unsupported-syntax pattern from pandas-dev/pandas.

```python
def is_hashable(obj: object, allow_slice: bool = True) -> TypeGuard[Hashable]:
    if allow_slice is False:
        if isinstance(obj, tuple) and any(isinstance(v, slice) for v in obj):
            return False
        elif isinstance(obj, slice):
            return False
    try:
        hash(obj)
    except TypeError:
        return False
    else:
        return True
```

Accepted evidence on True	hash(obj) succeeds and, when allow_slice=False, obj is neither a slice nor a tuple containing any slice.
Unsupported forms	The guard mixes exception-sensitive control flow, existential quantification over tuple elements via any(... for ... in ...), and negative conditions.
Why extraction stalls	Capturing the accepted region would require reasoning about the success of an arbitrary operation, element-wise constraints over an unbounded tuple, and exclusion facts that do not collapse to a simple positive witness.
Why typing is hard	The target Hashable is justified here by operational behavior, not by a nominal or structural witness that ordinary Python typing can express directly.

Fig. 12. Unsupported constraint syntax in pandas-dev/pandas.

Case Study: Unsupported Constraint Syntax in pandas-dev/pandas. The predicate is_hashable in pandas-dev/pandas shows why *guards with unsupported constraint syntax* is not merely a backlog category of unsupported constructs. Its logic establishes the narrowing claim by observing dynamic behavior under hash, combined with negative conditions over slice-shaped inputs. That is operationally meaningful code, but it does not package its evidence as the kind of positive, local witness that either PPG or a standard Python typechecker can consume directly. This is why *guards with unsupported constraint syntax* is not just a temporary implementation gap. Even a much stronger checker would require non-trivial typechecking features to internalize such guards faithfully. These syntaxes, when used for narrowing, point to an expressivity boundary.

The remaining class, *guards with runtime non-local dependencies*, is less widespread but still important. In these cases, the truth of the predicate depends on state external to the guarded value, such as instance state in a method predicate or surrounding module state that is not fixed as a constant. This pattern appears in only 3 source-level families, but each family is dense: the six equality-related predicates in python/mypy (`mypyc/ir/rtypes.py`), four schema-related predicates in pydantic/pydantic (`pydantic/_internal/_core_utils.py`), and `RaisesExc.matches` in `pytest-dev/pytest`. These cases show a different boundary of effective validation: even when the local syntax is simple, local path extraction becomes inadequate once narrowing depends on context outside the guarded argument itself.

MISS Case Study. Representative runtime non-local dependence from `pytest-dev/pytest`.

```
class RaisesExc(...):
    def matches(
        self, exception: BaseException | None
    ) -> TypeGuard[BaseExcT]:
        if exception is None:
            return False
        if not self._check_type(exception):
            return False
        if not self._check_match(exception):
            return False
        return self._check_check(exception)

    def _check_type(self, exception: BaseException) -> TypeGuard[BaseExcT]:
        self._fail_reason = _check_raw_type(self.expected_exceptions, exception)
        return self._fail_reason is None
```

Guarded argument	`exception` is the value being narrowed.
External state	The truth of the predicate depends on receiver configuration such as `self.expected_exceptions`, `self.match`, and `self.check`.
Why extraction stalls	Two calls with the same `exception` but different `RaisesExc(...)` objects can narrow differently, so there is no accepted region describable over `exception` alone.
Why this matters	The narrowing claim is a relation between the argument and ambient object state. Local path extraction over the formal parameter misses precisely the evidence that determines acceptance.

Fig. 13. Runtime non-local dependence in `pytest-dev/pytest`.

Case Study: Runtime Non-Local Dependencies in `pytest-dev/pytest`. The method `RaisesExc.matches` in `pytest-dev/pytest` shows the clearest receiver-state form of runtime non-local dependence. The narrowed value is `exception`, but whether the predicate returns `True` depends on configuration stored on the receiver object, which is impossible to know statically. The missing evidence is therefore not hidden in helper syntax or unsupported control flow, but in runtime object state outside the guarded argument. This receiver-state case is not isolated. The six-predicate equality cluster in python/mypy's `mypyc/ir/rtypes.py` shows the same relational shape, while pydantic/pydantic provides a module-state variant through mutable classification tables such as `_CORE_SCHEMA_FIELD_TYPES`, which could be modified at runtime. These guards are often perfectly

reasonable for application logic, but they rely on context that could only be made meaningful at runtime.

Overall, RQ2 shows that MISS cases are not random leftovers. They reflect recurring ways real `TypeGuard` predicates are written: by reusing opaque helpers, by expressing evidence through syntax whose meaning exceeds the positive local witnesses that Python typing handles well, or by relying on runtime non-local state. These styles are often natural from a software engineering perspective, but they produce narrowing evidence that is substantially harder to audit than the checker's trust model suggests.

RQ2 Result: Most MISS outcomes arise from recurring patterns rather than tool failure: helper-dependent guards dominate with over half of the cases (55.8%), while unsupported constraint syntax and runtime non-local dependence account for most of the rest.

5.3 RQ3: Kinds of Witnessed Unsoundness

Our third research question focuses on the FAIL outcomes: when the audit succeeds in generating and checking accepted inputs, what kinds of unsoundness does it uncover?

Among the 62 predicates for which the audit completes without MISS, 11 (17.7%) yield counterexamples. This is a high rate for a semantic audit technique applied to real open-source code, and it shows that the directly auditable portion of the ecosystem is not merely a collection of trivial or already-correct predicates. Instead, it contains a substantial density of narrowing claims that can be falsified automatically.

The 11 FAIL cases fall into 5 classes. The largest is *weak discriminators* with 4 cases (36.4%), followed by *duck-typed nominal claims* with 3 cases (27.3%), and *container kind mismatches* with 2 cases (18.2%). The remaining two classes each account for 1 case (9.1%): *shallow protocol checks* and *class-spoofing mismatches*.[3]

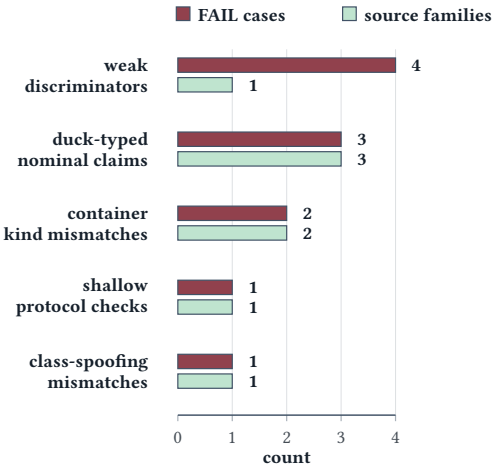

Fig. 14. Two views of the FAIL taxonomy.

A family-level view again helps avoid clone inflation. The 11 failing rows collapse to 8 source-level families when same-source same-class siblings are grouped together. In this deduplicated view, *duck-typed nominal claims* becomes the largest class with 3 families, *container kind mismatches* contributes 2 families, and *shallow protocol checks*, *class-spoofing mismatches*, and *weak discriminators* each contribute 1 family, with the four sibling black failures collapsed into the last class. This confirms that the counterexamples are not all variants of one bug. The audited failures span multiple independent semantic mismatch patterns.

These classes share a common semantic pattern: a relatively weak runtime witness is being used to justify a stronger static conclusion. In *weak discriminators*, a single attribute is treated as if it uniquely identifies a target subtype, even though the same attribute value can arise in

[3]For reproducibility, these correspond to the internal audit labels `NON_UNIQUE_DISCRIMINATOR_ATTR`, `NOMINAL_TARGET_DUCKTYPE_ACCEPTANCE`, `CONTAINER_KIND_MISMATCH`, `DUCK_ATTR_PROTOCOL_SHALLOW_CHECK`, and `ISINSTANCE_CLASS_SPOOFING`.

other cases. In *container kind mismatches*, a predicate checks a property of elements or iterable behavior but claims a stronger container-level type than the runtime evidence warrants. In *shallow protocol checks*, the presence of a small set of attributes is taken as sufficient evidence for a richer protocol-like or file-like claim. In *duck-typed nominal claims*, structural evidence is used to justify a nominal type claim. In *class-spoofing mismatches*, runtime class identity checks interact badly with objects that can imitate or spoof the expected class relationship.

These classes point to a recurring engineering mistake in `TypeGuard` design: developers often write predicates that are convenient and capture "something close to the intended concept," but are not strong enough in terms of proof obligations for the static narrowing they promise, i.e. "membership in the full target type." The result is unsound narrowing. Next, we proceed with concrete cases to show how these patterns manifest in code.

FAIL Case Study. Representative weak discriminator pattern from `psf/black`.

```
NL = Union["Node", "Leaf"]

class Node(Base):  # intended invariant: type >= 256
    ...
    self.type = type

class Leaf(Base):  # intended invariant: 0 <= type <= 256
    ...
    self.type = type

def is_lpar_token(nl: NL) -> TypeGuard[Leaf]:
    return nl.type == token.LPAR
```

Inferred on `True`	Let k be the token code compared by the predicate. The extracted path constraint is only `nl.type` = k; it does *not* establish that `nl` is a `Leaf`.
Generated witness	`nl = Node(k, [])` (concrete failing run: `Node(2, [])`)
Why it passes	The discriminator inspects only a shared field. Both `Node` and `Leaf` carry `.type`, so any `NL` object with `type` = k is accepted.
Why it fails the target	The claimed target is the nominal type `Leaf`, but the witness is a `Node`. This makes the check a *non-unique discriminator*: attribute equality is sufficient to return `True`, yet too weak to justify narrowing to `Leaf`. Even if constructor discipline is intended to separate these cases, the guard itself does not prove that separation at the call boundary.

Fig. 15. Weak discriminators in `psf/black`.

Case Study: Weak Discriminators in `psf/black`. A family of failures in `psf/black` illustrates the danger of non-unique discriminators. These predicates appear to rely on a shallow token attribute as if it uniquely identifies the intended token subtype. Our audit produced counterexamples showing that the chosen witness is not injective over the relevant runtime space: a value may satisfy the discriminator while still failing to inhabit the declared target type. The checker, however, narrows as if the discriminator were definitive. This family is useful because it makes the mismatch especially clear: the predicate body looks simple, plausible and intentional, yet its narrowing claim is stronger than the runtime evidence.

FAIL Case Study. Representative shallow protocol pattern from `ipython/ipython`.

```python
from collections.abc import Iterator

def _is_iterator(value: Any) -> TypeGuard[Iterator]:
    return hasattr(value, "__next__")
```

Inferred on `True`	The extracted path constraint is only that `value` has an attribute named `__next__`; it does *not* establish iterator protocol membership or class-level special-method support.
Generated witness	`value = FakeIter()` (with instance `value.__next__ = None`)
Why it passes	`hasattr` follows ordinary attribute lookup, so an instance attribute named `__next__` is enough for the guard to return `True`.
Why it fails the target	The target is `Iterator`, but operations such as `next(value)` rely on special-method lookup through the class, not an arbitrary instance attribute. The witness therefore passes the guard while remaining outside the promised iterator type.

Fig. 16. Shallow protocol evidence in `ipython/ipython`.

FAIL Case Study. Representative nominal-claim pattern from `Rapptz/discord.py`.

```python
def is_flag(obj: Any) -> TypeGuard[Type[FlagConverter]]:
    return hasattr(obj, "__commands_is_flag__")
```

Inferred on `True`	The extracted path constraint is only that `obj` exposes the field named `__commands_is_flag__`; it does *not* establish that `obj` is a class object or a subclass of `FlagConverter`.
Generated witness	`obj = NotFlag` (with `NotFlag.__commands_is_flag__ = None`)
Why it passes	The guard accepts any object or class carrying the guard attribute; it never checks for a nominal relation.
Why it fails the target	The target is the nominal type `Type[FlagConverter]`, but the witness is an unrelated class object. The sibling predicate `is_cog` has the same shape: `__cog_commands__` is structural evidence, not proof of membership in `Cog`.

Fig. 17. Structural evidence used for nominal claims in `Rapptz/discord.py`.

Case Study: Shallow Protocol Evidence in `ipython/ipython`. The predicate `_is_iterator` in `ipython/ipython` illustrates a shallow protocol check. It uses the presence of an iterator-like attribute to justify the stronger claim that the value is an `Iterator`. This is convenient and idiomatic in Python, but too weak for sound narrowing. In particular, `Iterator` is defined by a class-level `__next__` method, whereas the predicate checks only for an attribute named `__next__` on the instance itself. An object can therefore be made to look iterator-like, e.g., by setting `obj.__next__ = 0`, without actually being an `Iterator`. This case shows that a `TypeGuard` cannot treat a shallow duck-typing cue as sufficient for protocol membership.

Case Study: Structural Evidence Used for Nominal Claims in `Rapptz/discord.py`. The predicates `is_cog` and `is_flag` in `Rapptz/discord.py` show a related mismatch: structural checks used to justify nominal narrowing. The predicates accept values based on duck-typed properties compatible with the target, while the annotation claims membership in a nominal class or hierarchy. The checker then narrows as if that nominal relationship had been established. A developer may view such values as "good enough" for immediate use, but that is weaker than entitlement to nominal narrowing and may break if downstream code later adds an `isinstance` check.

FAIL Case Study. Representative container mismatch from `streamlit/streamlit`.

```python
def is_list_like(obj: object) -> TypeGuard[Sequence[Any]]:
    if isinstance(obj, str):
        return False
    if isinstance(obj, (list, set, tuple)):
        return True
    return isinstance(obj, (..., frozenset, KeysView, ...))
```

Inferred on `True`	The extracted path constraint is only that `obj` belongs to a whitelist of accepted runtime container classes; it does *not* establish `Sequence` semantics.
Generated witness	`obj = set()` (similarly: `frozenset()`)
Why it passes	The fast path explicitly accepts `set`, so the guard returns `True` without proving anything about positional sequence behavior.
Why it fails the target	The target is `Sequence[Any]`, which promises ordered, index-based access. A `set` in Python is not a `Sequence`. The predicate therefore proves a broader container notion than the annotation advertises.

Fig. 18. Container mismatches in `streamlit/streamlit`: a "list-like" runtime whitelist is broader than `Sequence[Any]`.

Case Study: Container Mismatches in `streamlit/streamlit`. One failure in `streamlit/streamlit` shows that the problem is not limited to shallow attribute checks: a `TypeGuard` can establish broad container semantics while still overclaiming a narrower target type. The predicate `is_list_like` checks membership in several container classes and then claims the input is a `Sequence`, but the accepted `set` type is actually wider than the `Sequence` protocol. The mistake is again overclaiming: sharing many properties with the target container kind does not justify the exact type promised to the checker, yet this is a mistake that could be easily committed by the developer.

Case Study: Class-Identity Mismatches in `reflex-dev/reflex`. A different failure mode appears in predicates such as `is_computed_var` in `reflex-dev/reflex`, where the body uses class-based checks but the claimed target type does not follow from them. In this case, the mismatch is aggravated by `__class__` spoofing, which makes the value appear to satisfy the expected runtime class relation. The core issue is that `isinstance`-style checks justify only the specific class relation they establish. If the predicate checks for class `A` but narrows to class `B`, the narrowing is semantically unjustified. Python's runtime flexibility makes this gap concrete: spoofed objects can pass the check without genuinely inhabiting the advertised target type.

Across the case studies, the common pattern is overclaiming: the runtime body establishes some useful operational evidence, but the `TypeGuard` annotation promotes that evidence into a stronger static claim than it can soundly support.

FAIL Case Study. Representative class-identity mismatch from `reflex-dev/reflex`.

```python
class FakeComputedVarBaseClass(property): ...

def is_computed_var(obj: Any) -> TypeGuard[ComputedVar]:
    return isinstance(obj, FakeComputedVarBaseClass)

class ComputedVar(...):
    @property
    def __class__(self) -> type:
        return FakeComputedVarBaseClass
```

Inferred on `True`	The extracted path constraint that leads to a `True` return value is only `isinstance(obj, FakeComputedVarBaseClass)`; it does *not* establish that `obj` is a `ComputedVar`.
Generated witness	`obj = FakeComputedVarBaseClass()`
Why it passes	The witness is genuinely an instance of the marker class used by the guard, so `isinstance(obj, FakeComputedVarBaseClass)` succeeds.
Why it fails the target	The target is the nominal type `ComputedVar`, but the witness is only the spoofable marker class. Overriding `ComputedVar.__class__` makes real computed vars satisfy the runtime test, yet the same check also accepts plain marker objects that do not inhabit the advertised target type.

Fig. 19. Class-identity mismatches in `reflex-dev/reflex`.

Overall, RQ3 shows that once effective auditing becomes possible, concrete unsoundness is not rare. The audit does not merely certify easy cases; it exposes a meaningful set of reviewed failures that illuminate how runtime checks and static claims come apart in practice.

RQ3 Result: Unsoundness is common in successful audits: (17.7%) auditable predicates yield counterexamples, typically because weak runtime evidence is used to justify a stronger static claim.

6 DISCUSSION

Our results suggest that `TypeGuard` is not merely a convenient typing feature, but a source of narrowing claims whose reliability is often weakly justified. This problem is broader than isolated bugs. Some predicates are directly unsound: they accept values outside the declared target type. Others resist practical automated validation, yet their results are still trusted by the checker. Both phenomena widen the gap between the static effect of narrowing and the support available to justify it.

This has implications for both tool builders and library authors. For tool builders, the MISS taxonomy points to useful warning or linting heuristics: helper-heavy predicates, predicates involving non-local coupling, and predicates relying on dynamic inspection could be flagged as hard to validate or treated more conservatively. The FAIL taxonomy likewise exposes recognizable unsoundness patterns that could inform diagnostics and linting.

For library authors, the study suggests that writing a good `TypeGuard` is not just about operational convenience. Its acceptance condition should be strong enough to justify the narrowing claim. Our MISS and FAIL taxonomies suggest several practical rules: prefer strong discriminators over partial

tags, justify nominal targets with nominal checks, justify container claims with container-level checks, and split complex logic into smaller, more auditable predicates when possible. More broadly, `TypeGuard` predicates should be written to support validation, not merely to enable narrowing. Since PPG already implements an automated audit pipeline, it could be integrated into development workflows such as continuous integration.

More broadly, our findings sharpen the discussion around gradual typing in dynamic languages. A common defense of Python typing mechanisms is that they intentionally balance soundness and practicality, but `TypeGuard` reveals where that balance relies on unchecked assumptions. It delegates part of the narrowing judgment to ordinary user code, while the corresponding validation infrastructure remains limited. The result is that narrowing claims are often accepted on weaker grounds than the type system presentation suggests.

Finally, the study suggests a practical engineering principle: when user code is allowed to drive static narrowing, auditability should be treated as a design concern. This does not require complete formal verification, but it does require recognizing that some styles of predicate construction produce stronger and more inspectable evidence than others. If future tooling or language design can encourage such styles, the validation gap around `TypeGuard` could be narrowed substantially.

7 THREATS TO VALIDITY

In this section, we discuss potential threats to the validity of the empirical audit.

PASS is not proof. In this study, PASS means only that the audit found no counterexample under the available search budget and the limited supported paths. It does not imply semantic correctness, completeness of exploration, or full alignment between runtime and static notions of the target type. We therefore interpret PASS conservatively throughout the paper.

Dataset scope. Our dataset is relatively small given the widespread use of Python. Part of the problem is that not all projects declare types, and among these, few declare `TypeGuards`. While our initial search found over 60,000 potential Python projects (Section 3), less than 1 % of projects passed a simple text-based search for `TypeGuards` and only 256 actually contained `TypeGuard` annotations in their source code (as opposed to comments or library code). Another problem is that our analysis pipeline, while automated, still requires non-trivial repository-level setup. This is the inherent limit of test-based audits: Each project needed a compatible dependency environment so that the generated test modules could import and execute the code under test. Thus we study the most popular 19 projects in the end. Despite the small size of the corpus, the projects contain over 100 `TypeGuard` declarations and uncover a variety of issues related to type soundness and the difficulty of typechecking user-defined predicates.

Repository concentration. In the evaluated set, `python/mypy` contributes a large share of the predicates and an even larger share of the PASS outcomes. This concentration can influence global outcome proportions. We partly address this by reporting repository-level distributions and a simple sensitivity view excluding `python/mypy`, but the dataset remains uneven.

Family duplication. Some bugs appear as multiple sibling predicates in the same source file. Raw counts can therefore overstate the prevalence of a semantic pattern if interpreted too literally. To mitigate this, we also discuss source-level family counts for both MISS and FAIL.

Taxonomy subjectivity. The reviewed classes assigned to FAIL and MISS cases reflect human judgment. While the labels are intended to summarize the dominant semantic issue in each predicate, some cases could plausibly fit multiple categories. In such cases, we manually assign the most explicit pattern that contributes to the outcome.

Method bias. The audit is better at predicates whose semantics can be localized and exercised through the extracted constraints. Predicates that rely on rich external state, deep helper chains, or dynamic inspection are less likely to yield complete auditing evidence. We treat this not merely as a weakness of the method, but rather as a valuable finding: many `TypeGuard` predicates resist effective audit. However, it remains a methodological limitation that shapes what kinds of evidence we can recover.

8 RELATED WORK

Our work draws inspiration from several sources. *Gradual typing* laid the foundation for types in Python [42, 46, 53], inspiring tools such as Mypy that encourage developers to write `TypeGuard` annotations. Typed Racket popularized type guards [47, 49] and furthermore demonstrated how a form of typechecking called *occurrence typing* could validate user-defined guards [48]. In the future, Python may wish to use occurrence typing; however, it would require cross-cutting changes to the typechecker. An alternative to occurrence typing that is equally expressive in theory, though harder to implement efficiently, is to use set-theoretic types [5–7]. In our opinion, integrating type guards into the typechecker would present a barrier to `TypeGuard` usage; thus PPG adapts ideas from *property-based testing* (PBT). Early on in the development of PPG, we were inspired by Goldstein and Pierce [17] to consider `TypeGuard` predicates as parsers and systematically derive generators from them. Currently we use Hypothesis to generate property-based tests [25]. In future work, we would like to integrate some of the many ongoing works that enhance PBT efficiency, e.g., [13, 34, 45, 55]. There have been several corpus studies on the adoption of gradual types in, e.g., Python, JavaScript, R, and Ruby [11, 12, 14, 15, 22, 24, 38, 51]. Adjacent empirical work studies how developers use dynamic-language features and typing-related practices in Smalltalk and Python [4, 9, 33]. Recent work further links gradual typing to missing runtime validation and type-confusion risks [50], and explores automated repair of Python static type errors [10]. These studies, and type-enforcement tools such as Scotty [21] and TPD [54], found many bugs in types and led us to believe (correctly, Section 5) that there would be bugs in type guards. Finally, we wish to acknowledge studies on the adoption of PBT [16, 18, 39], as they study a distinct but related problem in promoting rigorous testing tools. At a more human-facing level, Rawal et al. [40] study how hints, including test cases, help users find and fix Python bugs; this is orthogonal to our goal of synthesizing such tests automatically for `TypeGuard` predicates.

9 CONCLUSION

`TypeGuard` gives Python developers a flexible way to encode domain-specific refinement logic, but it shifts the burden of proof from the typechecker to user code. We introduced PPG, which combines static constraint extraction with property-based testing to synthesize and run semantic tests for existing `TypeGuard` predicates. Using PPG, we audited `TypeGuard` usage in a set of highly starred real-world Python repositories. We found that many predicates lack strong, testable evidence for their narrowing claims, and that many resist automated validation due to recurring structural patterns. Among the predicates that were tractable to audit, we found a non-trivial number of concrete counterexamples that refute the declared target types. These results expose a real validation gap and show that PPG is a practical tool for narrowing it.

Ultimately, this study highlights an important tension in gradual typing: as type systems add expressivity solely by trusting user-written narrowing claims, the necessity for automated semantic auditing becomes critical. PPG provides a practical step toward systematically verifying these claims. Our empirical results suggest that both developer practices and future typing infrastructure must evolve to better align the runtime evidence of type predicates with their static narrowing promises.

DATA-AVAILABILITY STATEMENT

We plan to submit an artifact that includes PPG, our dataset of Python projects, and scripts for reproducing our analysis.

REFERENCES

[1] Astral. 2025. ty Typechecker. https://docs.astral.sh/ty/, Accessed 2025-10-06.

[2] Gavin Bierman, Martin Abadi, and Mads Torgersen. 2014. Understanding TypeScript. In *ECOOP*. ACM, 257–281. doi:10.1007/978-3-662-44202-9_11

[3] Ambrose Bonnaire-Sergeant. 2019. *Typed Clojure in Theory and Practice*. Ph. D. Dissertation. Indiana University. https://hdl.handle.net/2022/23207

[4] Oscar Callaú, Romain Robbes, Éric Tanter, and David Röthlisberger. 2011. How Developers Use the Dynamic Features of Programming Languages: The Case of Smalltalk. In *MSR*. ACM, New York, NY, USA, 23–32. doi:10.1145/1985441.1985448

[5] Giuseppe Castagna, Guillaume Duboc, and José Valim. 2024. The Design Principles of the Elixir Type System. *The Art, Science, and Engineering of Programming* 8, 2 (2024). doi:10.22152/programming-journal.org/2024/8/4

[6] Giuseppe Castagna, Mickaël Laurent, and Kim Nguyen. 2024. Polymorphic Type Inference for Dynamic Languages. *Proceedings of the ACM on Programming Languages* 8, POPL (2024), 1179–1210. doi:10.1145/3632882

[7] Giuseppe Castagna, Mickaël Laurent, Kim Nguyen, and Matthew Lutze. 2022. On Type-Cases, Union Elimination, and Occurrence Typing. *Proceedings of the ACM on Programming Languages* 6, POPL (2022), 1–31. doi:10.1145/3498674

[8] Avik Chaudhuri, Panagiotis Vekris, Sam Goldman, Marshall Roch, and Gabriel Levy. 2017. Fast and Precise Type Checking for JavaScript. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 56:1–56:30. doi:10.1145/3133872

[9] Zhifei Chen, Yanhui Li, Bihuan Chen, Wanwangying Ma, Lin Chen, and Baowen Xu. 2020. An Empirical Study on Dynamic Typing Related Practices in Python Systems. In *ICPC*. ACM, New York, NY, USA, 83–93. doi:10.1145/3387904.3389253

[10] Yiu Wai Chow, Luca Di Grazia, and Michael Pradel. 2024. PyTy: Repairing Static Type Errors in Python. In *ICSE*. ACM, New York, NY, USA, 1–13. doi:10.1145/3597503.3639184

[11] Mark T. Daly, Vibha Sazawal, and Jeffrey S. Foster. 2009. Work In Progress: an Empirical Study of Static Typing in Ruby. In *PLATEAU*. 31–36. https://ecs.wgtn.ac.nz/foswiki/pub/Main/TechnicalReportSeries/ECSTR10-12.pdf

[12] Luca Di Grazia and Michael Pradel. 2022. The Evolution of Type Annotations in Python: An Empirical Study. In *FSE*. ACM, New York, NY, USA, 209–220. doi:10.1145/3540250.3549114

[13] Justin Frank, Benjamin Quiring, and Leonidas Lampropoulos. 2024. Generating Well-Typed Terms That Are Not "Useless". *Proceedings of the ACM on Programming Languages* 8, POPL, Article 77 (2024), 22 pages. doi:10.1145/3632919

[14] Zheng Gao, Christian Bird, and Earl T. Barr. 2017. To Type or Not to Type: Quantifying Detectable Bugs in JavaScript. In *ICSE*. IEEE, 758–769. doi:10.1109/ICSE.2017.75

[15] Aviral Goel and Jan Vitek. 2019. On the design, implementation, and use of laziness in R. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019), 153:1–153:27.

[16] Harrison Goldstein, Joseph W Cutler, Daniel Dickstein, Benjamin C Pierce, and Andrew Head. 2024. Property-based testing in practice. In *ICSE*. 1–13. doi:10.1145/3597503.3639581

[17] Harrison Goldstein and Benjamin C. Pierce. 2022. Parsing Randomness. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2, Article 128 (2022), 25 pages. doi:10.1145/3563291

[18] Harrison Goldstein, Jeffrey Tao, Zac Hatfield-Dodds, Benjamin C Pierce, and Andrew Head. 2024. Tyche: Making sense of PBT effectiveness. In *UIST*. 1–16. doi:10.1145/3654777.3676407

[19] Google. [n. d.]. Pytype Language. https://pypi.org/project/pytype

[20] Hanwen Guo and Ben Greenman. 2025. If-T: A Benchmark for Type Narrowing. *Programming* 10, 2 (2025), 17:1–17:31. doi:10.22152/programming-journal.org/2025/10/17

[21] Joshua Hoeflich, Robert Bruce Findler, and Manuel Serrano. 2022. Highly illogical, Kirk: spotting type mismatches in the large despite broken contracts, unsound types, and too many linters. *PACMPL* 6, OOPSLA2 (2022), 479–504. doi:10.1145/3563305

[22] Wuxia Jin, Dinghong Zhong, Zifan Ding, Ming Fan, and Ting Liu. 2021. Where to Start: Studying Type Annotation Practices in Python. In *ASE*. 529–541. doi:10.1109/ASE51524.2021.9678947

[23] Holger Krekel and pytest-dev team. 2015. pytest. https://docs.pytest.org/en/stable/index.html

[24] Xinrong Lin, Baojian Hua, Yang Wang, and Zhizhong Pan. 2023. Towards a Large-Scale Empirical Study of Python Static Type Annotations. In *SANER*. 414–425. doi:10.1109/SANER56733.2023.00046

[25] David R. MacIver and Alastair F. Donaldson. 2020. Test-Case Reduction via Test-Case Generation: Insights from the Hypothesis Reducer. In *ECOOP*. Schloss Dagstuhl, 13:1–13:27. doi:10.4230/LIPIcs.ECOOP.2020.13

[26] Petr Maj, Stefanie Muroya, Konrad Siek, Luca Di Grazia, and Jan Vitek. 2024. The Fault in Our Stars: Designing Reproducible Large-scale Code Analysis Experiments. In *ECOOP*, Vol. 313. Schloss Dagstuhl, 27:1–27:23. doi:10.4230/LIPICS.ECOOP.2024.27

[27] Meta. [n. d.]. Pyre Language. https://pyre-check.org

[28] Meta. 2023. flow-typed. https://flow-typed.github.io/flow-typed

[29] Meta. 2025. Pyrefly Typechecker. https://pyrefly.org/, Accessed 2025-10-06.

[30] Microsoft. [n. d.]. Pyright Language. https://github.com/Microsoft/pyright

[31] Microsoft. 2024. *TypeScript Documentation - Narrowing*. https://www.typescriptlang.org/docs/handbook/2/narrowing.html

[32] Guilherme Ottoni. 2018. HHVM JIT: a Profile-Guided, Region-Based Compiler for PHP and Hack. In *PLDI*. ACM, 151–165. doi:10.1145/3192366.3192374

[33] Yun Peng, Yu Zhang, and Mingzhe Hu. 2021. An Empirical Study for Common Language Features Used in Python Projects. In *SANER*. IEEE, 24–35. doi:10.1109/SANER50967.2021.00012

[34] Jacob Prinz and Leonidas Lampropoulos. 2023. Merging Inductive Relations. *Proceedings of the ACM on Programming Languages* 7, PLDI, Article 178 (2023), 20 pages. doi:10.1145/3591292

[35] Python. 2023. Typeshed. https://github.com/python/typeshed

[36] Python. 2025. Python Typing Council. https://github.com/python/typing-council, Accessed 2025-10-06.

[37] Python Team. 2025. typing - Support for Type Hints. https://docs.python.org/3/library/typing.html, Accessed 2025-10-08.

[38] Ingkarat Rak-amnouykit, Daniel McCrevan, Ana L. Milanova, Martin Hirzel, and Julian Dolby. 2020. Python 3 types in the wild: a tale of two type systems. In *DLS*. ACM, 57–70. doi:10.1145/3426422.3426981

[39] Savitha Ravi and Michael Coblenz. 2025. An Empirical Evaluation of Property-Based Testing in Python. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 3897–3923. doi:10.1145/3764068

[40] Ruchit Rawal, Victor-Alexandru Pădurean, Sven Apel, Adish Singla, and Mariya Toneva. 2024. Hints Help Finding and Fixing Bugs Differently in Python and Text-based Program Representations. doi:10.48550/arXiv.2412.12471 arXiv:2412.12471.

[41] Brianna M. Ren, John Toman, T. Stephen Strickland, and Jeffrey S. Foster. 2013. The Ruby Type Checker. In *SAC*. 1565–1572.

[42] Jeremy G. Siek and Walid Taha. 2006. Gradual Typing for Functional Languages. In *SFP. University of Chicago, TR-2006-06*. 81–92. http://scheme2006.cs.uchicago.edu/scheme2006.pdf

[43] Stripe. [n. d.]. Sorbet - A Static Type Checker for Ruby. https://sorbet.org/

[44] Mypy Team. [n. d.]. Mypy Language. http://www.mypy-lang.org

[45] Ryan Tjoa, Poorva Garg, Harrison Goldstein, Todd Millstein, Benjamin C. Pierce, and Guy Van den Broeck. 2025. Tuning Random Generators: Property-Based Testing as Probabilistic Programming. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2, Article 304 (2025), 28 pages. doi:10.1145/3763082

[46] Sam Tobin-Hochstadt and Matthias Felleisen. 2006. Interlanguage Migration: From Scripts to Programs. In *DLS*. 964–974. doi:10.1145/1176617.1176755

[47] Sam Tobin-Hochstadt and Matthias Felleisen. 2008. The Design and Implementation of Typed Scheme. In *POPL*. 395–406. doi:10.1145/1328438.1328486

[48] Sam Tobin-Hochstadt and Matthias Felleisen. 2010. Logical Types for Untyped Languages. In *ICFP*. 117–128. doi:10.1145/1863543.1863561

[49] Sam Tobin-Hochstadt, Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, Ben Greenman, Andrew M. Kent, Vincent St-Amour, T. Stephen Strickland, and Asumu Takikawa. 2017. Migratory Typing: Ten Years Later. In *SNAPL*. 17:1–17:17. doi:10.4230/LIPIcs.SNAPL.2017.17

[50] Dominic Troppmann, Aurore Fass, and Cristian-Alexandru Staicu. 2024. Typed and Confused: Studying the Unexpected Dangers of Gradual Typing. In *ASE*. ACM, New York, NY, USA, 1858–1870. doi:10.1145/3691620.3695549

[51] Alexi Turcotte, Aviral Goel, Filip Krikava, and Jan Vitek. 2020. Designing types for R, empirically. *PACMPL* 4, OOPSLA (2020), 181:1–181:25. doi:10.1145/3428249

[52] Guido van Rossum, Jukka Lehtosalo, and Łukasz Langa. [n. d.]. PEP 484 Type Hints. https://www.python.org/dev/peps/pep-0484

[53] Michael M. Vitousek, Andrew Kent, Jeremy G. Siek, and Jim Baker. 2014. Design and Evaluation of Gradual Typing for Python. In *DLS*. 45–56.

[54] Jack Williams, J. Garrett Morris, Philip Wadler, and Jakub Zalewski. 2017. Mixed Messages: Measuring Conformance and Non-Interference in TypeScript. In *ECOOP*. 28:1–28:29.

[55] Zhe Zhou, Ashish Mishra, Benjamin Delaware, and Suresh Jagannathan. 2023. Covering All the Bases: Type-Based Verification of Test Input Generators. *Proceedings of the ACM on Programming Languages* 7, PLDI, Article 157 (2023), 24 pages. doi:10.1145/3591271

[56] Jelle Zijlstra. 2024. *PEP 742 – Narrowing Types with TypeIs.* https://peps.python.org/pep-0742/